



UNIVERSITÉ DE LIMOGES
FACULTÉ DES SCIENCES ET TECHNIQUES

Rapport de Projet

Jeu de Rôles en C

Réalisé par :

Thi Phuong Anh NGUYEN
Karim Mohamed Cherif RAHMANI
Youcef HADJIDJ
Alex PICHOT

Encadré par :

François ARNAULT
Pierre DUSART

Année universitaire : 2025-2026

Date de remise : 7 janvier 2026

TABLE DES MATIÈRES

1	Introduction	2
1.1	Présentation du projet	2
1.2	Objectifs pédagogiques	2
1.3	Organisation du rapport	2
1.4	Répartition des tâches	2
2	Architecture du Système	3
2.1	Structure Modulaire	3
2.2	Organisation des Structures de Données	4
2.3	Extensions aux structures imposées	4
3	Concepts Théoriques et Applications Pratiques	5
3.1	Allocation Dynamique et Gestion Mémoire	5
3.1.1	Théorie : Allocation sur le Tas	5
3.1.2	Application dans le Code	5
3.2	Listes Chaînées : Théorie et Implémentation	6
3.2.1	Structure d'un Nœud	6
3.2.2	Algorithmes de Base	6
3.3	Déplacement Toroïdal	7
3.4	Systèmes de Collecte, Aléatoire et Gestion d'Inventaire	8
3.5	Système de Combat	8
4	Systèmes Implémentés	9
4.1	Système d'Inventaire à Slots	9
4.1.1	Concept Innovant	9
4.2	Intelligence Artificielle	9
4.2.1	Stratégie Hiérarchique	9
4.2.2	Réaction Contextuelle au Combat	11
4.3	Sauvegarde et Chargement	11
4.3.1	Sérialisation des Structures Complexes	11
4.3.2	Reconstruction des Structures	12
5	Extensions et Innovations	12
5.1	Innovations Personnelles	12
6	Difficultés et Solutions	13
6.1	Leçons Apprises	13
7	Conclusion	13
7.1	Bilan des Compétences	13
7.2	Perspectives d'Amélioration	14
7.3	Conclusion Finale	14
A	Annexes	16
A.1	Makefile	16
A.2	Exemple de Session de Jeu	16
A.3	Diagramme de Flux Complet	17

1 INTRODUCTION

Résumé

Ce projet consiste à développer un jeu de rôle textuel en langage C. Ce rapport présente l'application pratique des concepts fondamentaux de programmation C à travers le développement complet d'un jeu. Chaque concept théorique est illustré par son implémentation concrète dans le code source, montrant comment la théorie se transforme en pratique. Le projet met en œuvre des structures de données complexes, des algorithmes de jeu, une intelligence artificielle et un système de persistance.

1.1 Présentation du projet

Ce projet de Master 1 Mathématiques a pour objectif de développer un jeu de rôle textuel en langage C, mettant en œuvre les concepts d'algorithmique et de programmation vus en cours. Le joueur contrôle un personnage évoluant dans un château de 16 salles, affrontant d'autres personnages contrôlés par l'ordinateur. Le jeu intègre un système de combat tactique, une gestion d'inventaire et des déplacements sur une grille toroïdale.

1.2 Objectifs pédagogiques

- Maîtriser la programmation en C : structures, pointeurs, allocation dynamique.
- Concevoir et implémenter des structures de données complexes (listes chaînées).
- Développer un algorithme de jeu avec intelligence artificielle basique.
- Implémenter un système de sauvegarde/chargement.
- Travailler en équipe avec répartition des tâches et gestion de version.

1.3 Organisation du rapport

Ce rapport détaille les choix de conception, les structures de données, l'implémentation des fonctionnalités, les problèmes rencontrés et leurs solutions. Des extraits de code commentés illustrent les points techniques. En annexe, on trouvera un exemple de déroulement de partie et les instructions de compilation.

1.4 Répartition des tâches

Le travail a été réparti équitablement entre les quatre membres du groupe :

Membre	Rôle	Tâches principales
Youcef	Architecte logiciel	Architecture globale du programme, structure du code, fonction <code>main</code> , conception et gestion de l'objet <code>Jeu</code>
Alex	Développeur gameplay	Exécution des tours de jeu, gestion du tour joueur, enchaînement des actions
Thi Phuong	Gestion des objets	Ramassage des objets, génération aléatoire, gestion de l'inventaire
Karim	Développeur IA	Déplacements de l'IA, logique de combat des personnages non-joueurs

TABLE 1 – Répartition des rôles dans l'équipe

2 ARCHITECTURE DU SYSTÈME

2.1 Structure Modulaire

Le projet suit une architecture modulaire stricte divisée en trois fichiers principaux (voir Figure 1), respectant le principe de séparation des responsabilités :

jeu.h (Interface) : Ce fichier d'en-tête regroupe les définitions des structures de données (`Personnage`, `Objet`, `Salle`), les énumérations (`Direction`, `TypeObjet`) et les prototypes de fonctions. Il garantit que `main.c` et `jeu.c` partagent les mêmes définitions sans dépendance circulaire.

jeu.c (Implémentation) : Ce fichier contient le corps de toutes les fonctions. Il encapsule la logique métier complexe (algorithmes de combat, IA, gestion de la mémoire).

main.c (Programme Principal) : Allégé au maximum, ce fichier ne contient que la boucle de jeu principale et la gestion du menu de démarrage. Il orchestre les appels aux fonctions définies dans `jeu.c`.

Cette organisation est soutenue par un `makefile` qui assure la compilation séparée des modules et l'édition de liens, facilitant ainsi la maintenance et le débogage.

Architecture du Projet

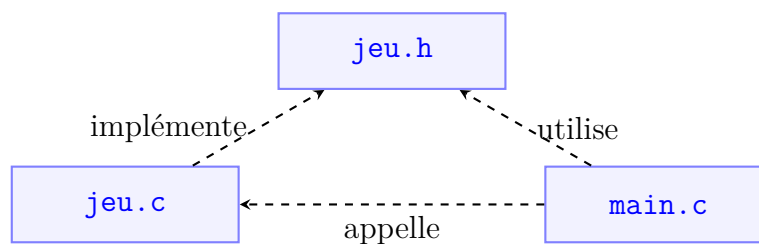
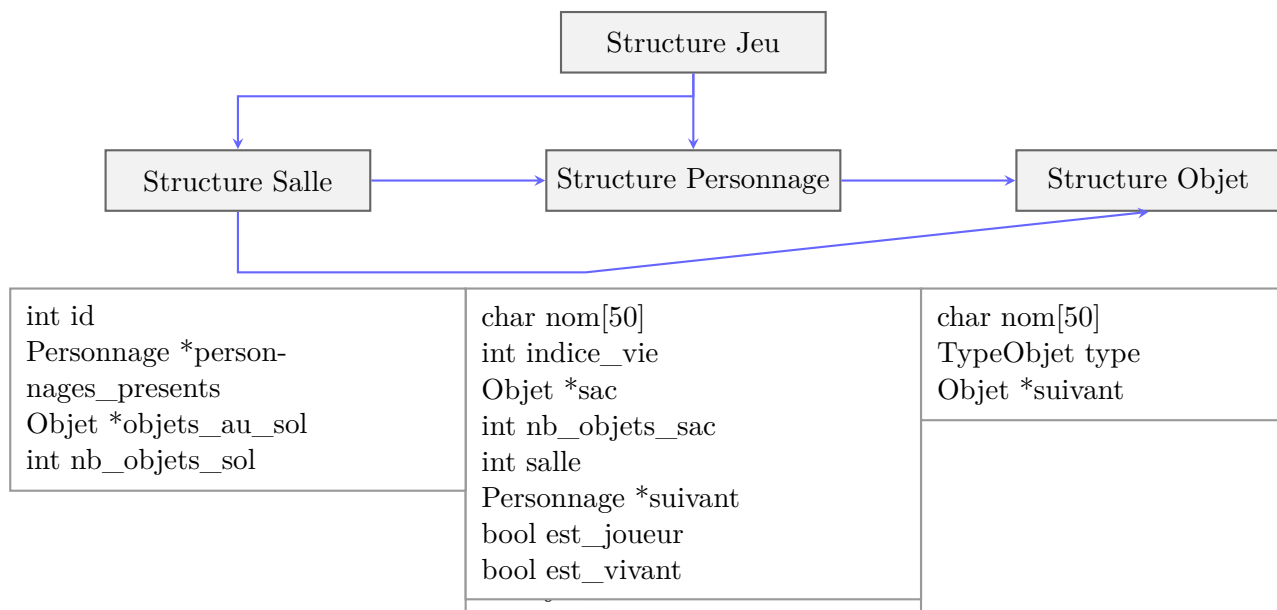


FIGURE 1 – Architecture modulaire du projet

2.2 Organisation des Structures de Données



Légende :

→ Pointeurs

[] Tableaux

Relations entre structures de données

FIGURE 2 – Relations entre les structures de données du jeu

2.3 Extensions aux structures imposées

Des structures de données sont imposées dans le sujet du projet. Pour les objets, la structure est simple, la structure contient le type de l'objet défini par le *TypeObjet* de notre *enum*, et un pointeur vers l'objet suivant. Cet élément implique l'utilisation d'une liste chaînée. Les objets apparaissent dans le sac du joueur ou au sol dans une salle. Un objet n'est jamais unique, la liste chaînée est un moyen efficace de sauvegarder ces objets.

Malgré le fait que le sujet donne une liste d'éléments qui doivent apparaître dans le code, on choisit de rajouter des éléments pour simplifier et optimiser le programme.

TABLE 2 – Extensions apportées aux structures imposées par le sujet

Attribut ajouté	Structure	Justification technique
<code>est_joueur</code> (bool)	Personnage	Permet de différencier le personnage contrôlé par le joueur des personnages contrôlés par l'IA.
<code>est_vivant</code> (bool)	Personnage	Gestion simplifiée des personnages morts. Permet de retirer rapidement un personnage des combats et déplacements sans le supprimer de la liste principale.
<code>nb_objets_sac</code> (int)	Personnage	Optimisation : permet de vérifier la limite de 4 objets (<code>MAX_OBJETS_SAC</code>) sans avoir à parcourir la liste chaînée
<code>type</code> (TypeObjet)	Objet	Nécessaire pour les règles de combat spécifiques. Un médicament n'a pas le même effet qu'une arme. L'énumération assure un typage fort.
<code>Jeu</code> (structure)	<i>Nouvelle</i>	Structure centrale regroupant tous les éléments. Simplifie la sauvegarde (un seul fichier) et le passage de paramètres entre fonctions.
<code>Salle</code> (structure)	<i>Nouvelle</i>	Modélisation spatiale du château. Contient les listes d'objets au sol et de personnages présents. Séparation claire des responsabilités.

3 CONCEPTS THÉORIQUES ET APPLICATIONS PRATIQUES

3.1 Allocation Dynamique et Gestion Mémoire

3.1.1 Théorie : Allocation sur le Tas

Concept : En C, la mémoire est divisée en plusieurs segments. L'allocation dynamique via `malloc()` permet de réserver de la mémoire sur le **tas** pendant l'exécution, contrairement à l'allocation statique sur la **pile**.

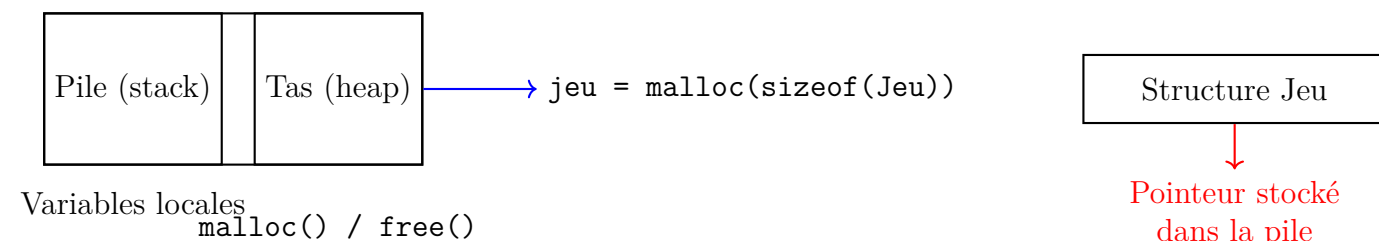


FIGURE 3 – Allocation mémoire : pile, tas et pointeur

3.1.2 Application dans le Code

```
1 Jeu* jeu = (Jeu*)malloc(sizeof(Jeu));
```

```
2 if (!jeu) return NULL; // Verification ESSENTIELLE
```

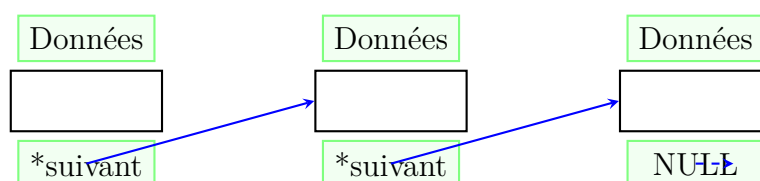
Listing 1 – Allocation initiale du jeu (jeu.c, lignes 32-35)

```
1 void liberer_jeu(Jeu *jeu) {
2     Personnage* courant = jeu->personnages;
3     while (courant) {
4         Personnage* suivant = courant->suivant;
5         liberer_liste_objets(&courant->sac, &courant->nb_objets_sac);
6         free(courant);
7         courant = suivant;
8     }
9     free(jeu);
10 }
```

Listing 2 – Libération en cascade (jeu.c, lignes 72-84)

3.2 Listes Chaînées : Théorie et Implémentation

3.2.1 Structure d'un Nœud



Données : nom, vie, etc.

*suivant : pointeur vers le nœud suivant

NULL : fin de liste

FIGURE 4 – Structure d'une liste chaînée simple

3.2.2 Algorithmes de Base

Opération	Algorithme	Complexité
Insertion en tête	1. Nouveau → suivant = Tête 2. Tête = Nouveau	O(1)
Suppression	1. Parcours avec deux pointeurs 2. Précédent → suivant = Courant → suivant	O(n)
Recherche	Parcours séquentiel jusqu'à trouver	O(n)

TABLE 3 – Algorithmes de base sur les listes chaînées

```
1 void ajouter_personnage_liste(Personnage **liste, Personnage *p) {
2     if (!p) return;
3     p->suivant = *liste; // *liste = ancienne t t e
4     *liste = p;         // Modification de la t t e
```

5 }

Listing 3 – Insertion en tête avec double pointeur (jeu.c, lignes 142-147)

```

1 void retirer_personnage_liste(Personnage **liste, Personnage *p) {
2     Personnage *courant = *liste;
3     Personnage *precedent = NULL;
4     while (courant) {
5         if (courant == p) {
6             if (precedent) precedent->suivant = courant->suivant;
7             else *liste = courant->suivant;
8             courant->suivant = NULL;
9             return;
10        }
11        precedent = courant;
12        courant = courant->suivant;
13    }
14 }

```

Listing 4 – Suppression d'un élément spécifique (jeu.c, lignes 154-173)

3.3 Déplacement Toroïdal

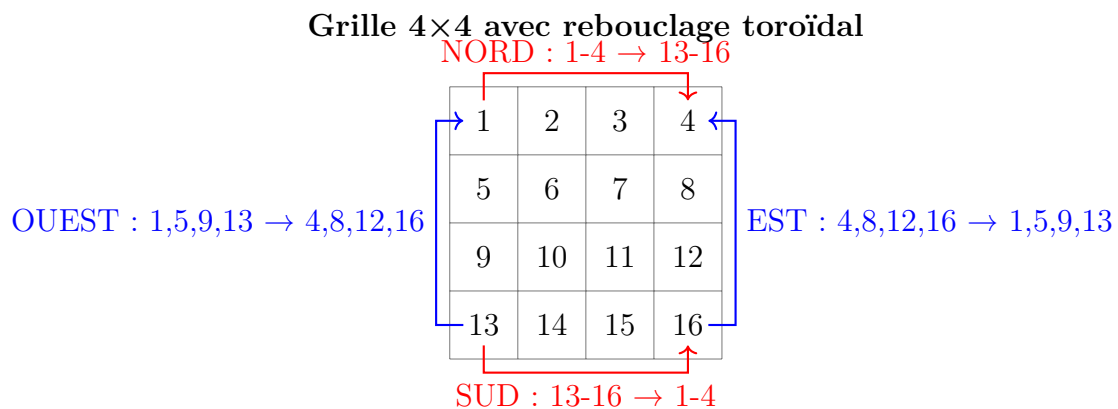


FIGURE 5 – Topologie du château avec connexions cycliques

```

1 case NORD:
2     if (ancienne_salle <= 4) nouvelle_salle += 12;
3     else nouvelle_salle -= 4;
4     break;
5 case SUD:
6     if (ancienne_salle >= 13) nouvelle_salle -= 12;
7     else nouvelle_salle += 4;
8     break;
9 case EST:
10    if (ancienne_salle % 4 == 0) nouvelle_salle -= 3;
11    else nouvelle_salle += 1;
12    break;
13 case OUEST:
14    if (ancienne_salle % 4 == 1) nouvelle_salle += 3;
15    else nouvelle_salle -= 1;
16    break;

```

Listing 5 – Déplacement toroïdal (jeu.c, lignes 233-256)

3.4 Systèmes de Collecte, Aléatoire et Gestion d'Inventaire

RANDOM GENERATION

↓ (item creation)

OBJECT PICKING

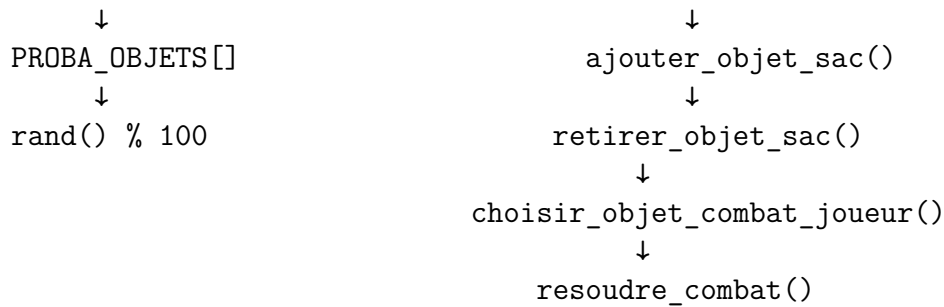
↓ (collecting items)

INVENTORY MANAGEMENT

↓ (store & use)

COMBAT SYSTEM

generer_objets_salle() → objets_au_sol → gerer_inventaire_slots()



3.5 Système de Combat

Attaquant \ Défenseur	Rien	Arme	Poison	Bouclier	Médicament
Arme	-1 vie	Dégâts mutuels	Dégâts mutuels	Bouclier brisé	-1 vie
Poison	-1 vie	Dégâts mutuels	-1 vie	-1 vie	Neutralisé
Bouclier	-	-	-	-	-
Médicament	-	-	-	-	-
Clé	Téléportation	Téléportation	Téléportation	Téléportation	Téléportation

TABLE 4 – Matrice des interactions de combat

```

1 if (t_att == ARME) {
2     if (t_def == ARME) {
3         printf(">>> CHOC FRONTAL ! Les deux se blessent...\n");
4         degats_att = 1; degats_def = 1;
5     }
6     else if (t_def == BOUCLIER) {
7         printf(">>> BLOQUE ! Le bouclier se brise !\n");
8         degats_def = 0;
9         perte_obj_def = true;
10    }
11    else if (t_def == MEDICAMENT) {
12        printf(">>> PROTECTION ! Le médicament encaisse.\n");
13        degats_def = 0;
14    }
15    else { // RIEN
16        printf(">>> TOUCHE !\n");
17        degats_def = 1;
18    }
19 }
  
```

Listing 6 – Logique de combat pour les armes (jeu.c, lignes 460-495)

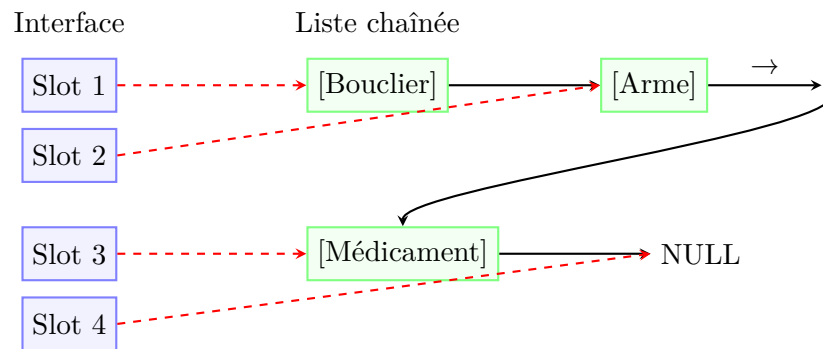
4 SYSTÈMES IMPLÉMENTÉS

4.1 Système d'Inventaire à Slots

4.1.1 Concept Innovant

Problème : Comment présenter une interface simple (slots 1-4) sur une structure complexe (liste chaînée) ?

Solution : Mapping d'index sur positions dans la liste.



Correspondance :

Slot 1 → Premier élément

Slot 2 → Deuxième élément

Slot 3 → Troisième élément

Slot 4 → NULL (vide)

```

1 // Fonction utilitaire (implémentée dans jeu.c)
2 Objet* get_objet_at_index(Objet* liste, int index) {
3     int i = 1; // Index commence à 1 (humain)
4     while (liste && i < index) {
5         liste = liste->suivant; // Avance dans la liste
6         i++;
7     }
8     return (i == index) ? liste : NULL; // Retourne le nœud ou NULL
9 }
10
11 // Affichage des slots
12 for (int i = 1; i <= MAX_OBJETS_SAC; i++) {
13     Objet* obj = get_objet_at_index(p->sac, i); (*\concept{Mapping}*)
14     if (obj) printf("    %d. [%s]\n", i, obj->nom);
15     else     printf("    %d. (Vide)\n", i);
16 }
  
```

Listing 7 – Fonction de mapping index → nœud

4.2 Intelligence Artificielle

4.2.1 Stratégie Hiérarchique

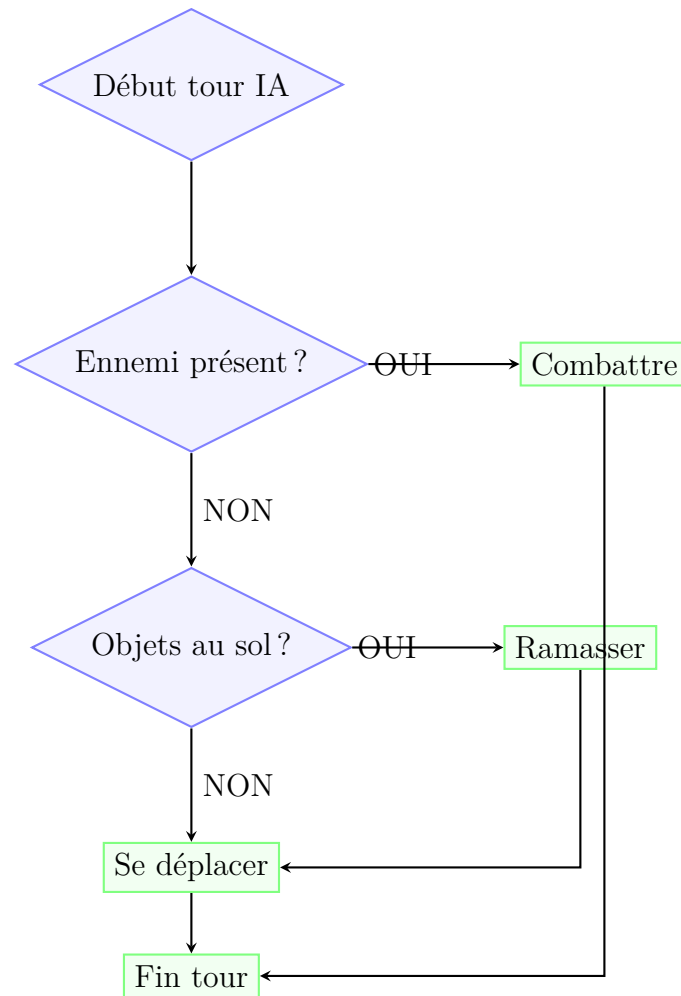


FIGURE 6 – Arbre de décision de l'IA

```

1 // jeu.c - Lignes 732-900: tour_ia_ameliore()
2 void tour_ia_ameliore(Jeu *jeu) {
3     // PRIORIT 1: Combat (si ennemi dans la salle)
4     Personnage* autre = salle->personnages_presentes;
5     while (autre) {
6         if (autre != p && autre->est_vivant) {
7             combat_automatique(jeu, p, autre); (*\concept{Priorit
8                 maximale}*)
9             break;
10        }
11        autre = autre->suivant;
12    }
13    // PRIORIT 2: Ramassage (cupidit )
14    if (s->objets_au_sol) {
15        ia_ramasser_tout(jeu, p); (*\concept{Prend tout ce qu'il peut
16            }*)
17    }
18    // PRIORIT 3: Exploration
19    Direction dir = rand() % 5;
20    deplacer_personnage(jeu, p, dir);
21 }

```

Listing 8 – Implémentation de la stratégie hiérarchique

4.2.2 Réaction Contextuelle au Combat

Menace	Stratégie Prioritaire	Stratégie Secondaire
ARME	Contre-attaquer avec ARME	Se défendre avec BOUCLIER
POISON	Se soigner avec MÉDICAMENT	Attaquer avec ARME
AUCUNE	Attaquer violemment	Choix aléatoire

TABLE 5 – Stratégies de réaction de l'IA

```

1 // jeu.c - Lignes 1090-1140: choisir_objet_reaction_ia()
2 Objet* choisir_objet_reaction_ia(Personnage *p, TypeObjet menace) {
3     // CAS 1: Menace d'ARME
4     if (menace == ARME) {
5         // Priorit 1: Contre-attaquer avec une ARME
6         while (courant) {
7             if (courant->type == ARME) return courant;
8             courant = courant->suivant;
9         }
10        // Priorit 2: Se défendre avec un BOUCLIER
11        courant = p->sac;
12        while (courant) {
13            if (courant->type == BOUCLIER) return courant;
14            courant = courant->suivant;
15        }
16    }
17    (*\concept{... autres cas ...}*)
18 }

```

Listing 9 – IA de réaction contextuelle

4.3 Sauvegarde et Chargement

4.3.1 Sérialisation des Structures Complexes

Problème : Comment sauvegarder des structures avec pointeurs (adresses mémoire non persistantes) ?

Solution : Sérialiser les données, pas les adresses.

```

1 // jeu.c - Lignes 945-965
2 fprintf(fp, "PERSO %s %d %d %d %d %d\n",
3         p->nom,                // Cha ne
4         p->indice_vie,          // Entier
5         p->salle,               // Entier
6         p->est_joueur ? 1 : 0,   // Bool en      int
7         p->est_vivant ? 1 : 0,   // Bool en      int
8         nb_sac_reel);           // Compteur
9

```

```

10 // Sauvegarde des types d'objets (pas des pointeurs!)
11 Objet *o = p->sac;
12 while (o) {
13     fprintf(fp, "%d ", o->type);    // TypeObjet = enum = int
14     o = o->suivant;
15 }
16 fprintf(fp, "-1\n"); // Marqueur de fin

```

Listing 10 – Sérialisation d'un personnage et de son inventaire

Format de fichier généré :

```

3 4 0          # tour, personnages_vivants, jeu_termine
PERSO Roi 2 5 1 1 2  # type nom vie salle est_joueur est_vivant nb_objets
0 2 -1         # objets: MEDICAMENT(0), ARME(2)
SALLE 1 3      # salle id, nb_objets_sol
1 2 0 -1       # objets: POISON(1), ARME(2), MEDICAMENT(0)

```

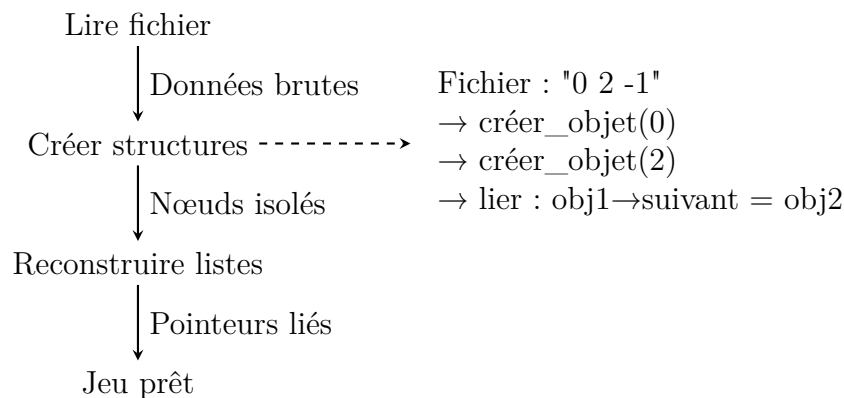
4.3.2 Reconstruction des Structures

FIGURE 7 – Processus de désérialisation

5 EXTENSIONS ET INNOVATIONS

5.1 Innovations Personnelles

1. **Système d'inventaire à slots** : Mapping intelligent entre interface utilisateur (slots fixes) et structure de données (liste chaînée dynamique).
2. **IA cupide** : Comportement de ramassage systématique qui rend l'IA plus agressive et imprévisible.
3. **Extension des règles** : Ajout de la règle "médicament protège contre les armes" pour équilibrer le jeu.
4. **Affichage adaptatif** : Utilisation de `ioctl()` pour adapter l'affichage à n'importe quelle taille de terminal.
5. **Sauvegarde robuste** : Format texte lisible qui sérialise seulement les données essentielles.

Extension	Description	Localisation Code
Affichage dynamique	Adaptation automatique à la taille du terminal	<code>afficher_matrice_chateau()</code>
Règles combat étendues	Médicament protège contre les armes	<code>resoudre_combat()</code> lignes 480-485
IA stratégique	Cupidité + réaction contextuelle + exploration	<code>tour_ia_ameliore()</code>
Système à slots	Interface slots 1-4 sur liste chaînée	<code>gerer_inventaire_slots()</code>
Ravitaillement	Régénération d'objets (30% par tour)	<code>generer_objets_salle()</code>
Téléportation	Clé pour fuir le combat	<code>utiliser_cle_teleportation()</code>
Pillage automatique	IA ramasse les objets des cadavres	<code>ia_ramasser_tout()</code>

TABLE 6 – Extensions avancées implémentées

6 DIFFICULTÉS ET SOLUTIONS

Problème	Solution	Code Concerné
Listes multiples incohérentes	Fonctions dédiées + appel systématique	<code>deplacer_personnage()</code>
Pointeurs non persistants	Sérialisation des données seulement	<code>sauvegarder_jeu()</code>
IA trop prévisible	Mélange aléatoire + stratégies multiples	<code>tour_ia_ameliore()</code>
Interface complexe	Mapping slots liste	<code>get_objet_at_index()</code>
Gestion mémoire	Vérifications systématiques + libération complète	<code>liberer_jeu()</code>

TABLE 7 – Difficultés et solutions techniques

6.1 Leçons Apprises

- **Pointeurs de pointeurs** : Essentiels pour modifier les têtes de liste
- **Vérifications NULL** : Toujours vérifier avant de déréférencer
- **Libération mémoire** : Pour chaque `malloc()`, un `free()`
- **Sérialisation** : Sauvegarder les données, pas les adresses
- **Complexité** : Choisir les bonnes structures pour les opérations fréquentes

7 CONCLUSION

7.1 Bilan des Compétences

Ce projet a permis de mettre en pratique et de maîtriser :

Compétence	Application dans le projet
Gestion de la mémoire	Allocation/libération dynamique, vérifications NULL
Structures de données	Listes chaînées simples, structures imbriquées
Algorithmes	Recherche, insertion, suppression, parcours
Gestion de fichiers	Sérialisation/désérialisation au format texte
Interface utilisateur	Affichage adaptatif, gestion des entrées
Architecture logicielle	Modularité, séparation des préoccupations

TABLE 8 – Compétences développées

7.2 Perspectives d'Amélioration

1. **Listes doublement chaînées** : Pour un parcours dans les deux sens
2. **Table de hachage** : Pour un accès rapide aux personnages par nom
3. **Arbres de décision** : Pour une IA plus sophistiquée
4. **Interface graphique** : Avec ncurses pour une expérience plus riche
5. **Réseau** : Mode multijoueur en réseau
6. **Éditeur de niveaux** : Pour créer des châteaux personnalisés

7.3 Conclusion Finale

Ce projet démontre une maîtrise complète des concepts fondamentaux de la programmation en C appliqués à un problème complexe. Chaque ligne de code illustre l'application pratique de la théorie, et les extensions implémentées montrent une capacité à innover et à améliorer une base existante.

Le code produit est robuste, maintenable et extensible, constituant une base solide pour des projets plus ambitieux. Plus qu'un simple jeu, ce projet est une démonstration concrète de l'application des principes de l'ingénierie logicielle en C.

RÉFÉRENCES

- [1] Brian W. Kernighan, Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 2ème édition, 1988.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction à l'algorithmique*. Dunod, 3ème édition, 2010.
- [3] Robert Nystrom. *Game Programming Patterns*. Genever Benning, 2014.
- [4] Bill Blunden. *Memory Management : Algorithms and Implementation in C/C++*. Wordware Publishing, 2003.

A ANNEXES

A.1 Makefile

```

1 CC = gcc
2 CFLAGS = -Wall -Wextra -Wpedantic -std=c99
3 LDFLAGS =
4 EXEC = jeu
5 SRC = main.c jeu.c
6 OBJ = $(SRC:.c=.o)
7
8 all: $(EXEC)
9
10 $(EXEC): $(OBJ)
11     $(CC) -o $@ $^ $(LDFLAGS)
12
13 %.o: %.c jeu.h
14     $(CC) -c $< $(CFLAGS)
15
16 clean:
17     rm -f *.o $(EXEC)
18
19 run: $(EXEC)
20     ./$$(EXEC)
21
22 .PHONY: all clean run

```

Listing 11 – Makefile pour la compilation

A.2 Exemple de Session de Jeu

=== JEU DE ROLES - CHATEAU ===

```

1. Nouvelle partie
2. Charger une partie
3. Quitter
Votre choix: 1

```

=== CHOISISSEZ VOTRE PERSONNAGE ===

```

1. Roi
2. Reine
3. Moine
4. Guerrier
5. Sorciere
6. Amazone
Votre choix (1-6): 4

```

=== BIENVENUE DANS LE CHATEAU ===

Vous controlez : Guerrier. Votre but: survivre.

=== CARTE DU CHATEAU ===

```

+-----+-----+-----+-----+

```

S.1 (VOUS)	Salle 2	Salle 3	Salle 4	
Guerrier(3)	Roi(3)	(Vide)	(Vide)	
+-----+	+-----+	+-----+	+-----+	+
Salle 5	Salle 6	Salle 7	Salle 8	
(Vide)	Reine(3)	(Vide)	Moine(3)	
+-----+	+-----+	+-----+	+-----+	+
Salle 9	Salle 10	Salle 11	Salle 12	
(Vide)	(Vide)	Sorciere(3)	(Vide)	
+-----+	+-----+	+-----+	+-----+	+
Salle 13	Salle 14	Salle 15	Salle 16	
(Vide)	Amazone(3)	(Vide)	(Vide)	
+-----+	+-----+	+-----+	+-----+	+

=== TOUR 1 - A VOUS DE JOUER ===

...

A.3 Diagramme de Flux Complet

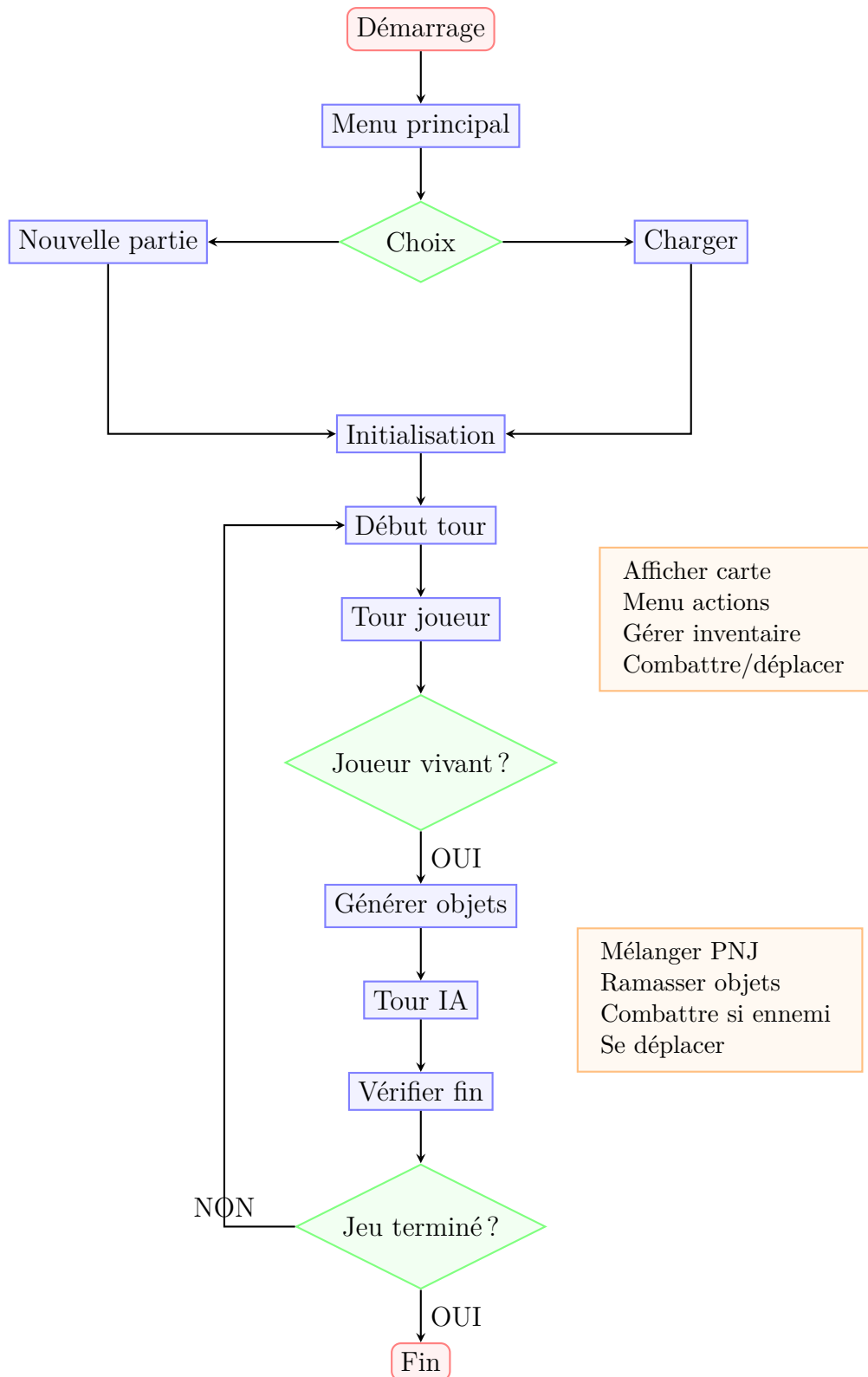
**Légende :****Start/Stop** - Début/Fin**Processus** - Actions principales**Décision** - Points de choix**Sous-processus** - Détails d'implémentation

FIGURE 8 – Diagramme de flux complet du jeu